Experiment-13

Name: -P. Akhila **Regno: -**192472320

Course Code: -MLA0305 **Slot-**B

Title

Advanced Policy Optimization using PPO, TRPO and DDPG

Aim

To implement and evaluate advanced policy optimization methods PPO, TRPO, and DDPG and compare their performance using reward, stability, variance, and training time.

Procedure

1. Initialize the Pendulum-v1 continuous-control environment.
2. Implement PPO (Proximal Policy Optimization) using Stable-Baselines3.
3. Implement TRPO (Trust Region Policy Optimization) using Stable-Baselines3 and SB3-Contrib.
4. Implement DDPG (Deep Deterministic Policy Gradient) with suitable continuous-action settings.
5. Train all three algorithms for the specified number of timesteps.
6. Record the episode rewards obtained during training.
7. Calculate the final average reward and reward variance for each algorithm.
8. Measure the training time of each algorithm.
9. Plot learning curves to compare the convergence behavior.
10. Generate bar charts for final reward, variance, and training time.
11. Generate a box plot to compare reward distributions.
12. Generate a pie chart to show relative final performance.

13. Prepare a summary table and save the complete training results as a CSV file.

Code

```
!pip -q install stable-baselines3 sb3-contrib gymnasium


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import gymnasium as gym
import time
from stable_baselines3 import PPO, DDPG
from sb3_contrib import TRPO
from stable_baselines3.common.monitor import Monitor
from stable_baselines3.common.callbacks import BaseCallback
from stable_baselines3.common.noise import NormalActionNoise


np.random.seed(42)


print("="*75)
print("EXPERIMENT 13")
print("Advanced Policy Optimization using PPO, TRPO and DDPG")
print("="*75)
```

```
ENV_NAME="Pendulum-v1"
TIMESTEPS=10000


class RewardCallback(BaseCallback):
    def __init__(self):
        super().__init__()
        self.rewards=[]
        self.steps=[]

    def _on_step(self):
        infos=self.locals.get("infos",[])
        dones=self.locals.get("dones",[])

        for info,done in zip(infos,dones):
            if done and "episode" in info:
                self.rewards.append(
                    float(info["episode"]["r"])
                )
                self.steps.append(
                    self.num_timesteps
```

```
            )
        return True

def make_env():
    return Monitor(gym.make(ENV_NAME))

print("\nEnvironment:",ENV_NAME)
print("Training Timesteps:",TIMESTEPS)

ppo_env=make_env()
ppo_callback=RewardCallback()

start=time.time()

ppo=PPO(
    "MlpPolicy",
    ppo_env,
    learning_rate=0.0003,
    n_steps=256,
    batch_size=64,
    gamma=0.99,
```

```
    verbose=0,
    seed=42
)

ppo.learn(
    total_timesteps=TIMESTEPS,
    callback=ppo_callback
)

ppo_time=time.time()-start
ppo_env.close()

print("PPO Completed")

trpo_env=make_env()
trpo_callback=RewardCallback()

start=time.time()

trpo=TRPO(
    "MlpPolicy",
```

```
    trpo_env,
    learning_rate=0.0003,
    gamma=0.99,
    verbose=0,
    seed=42
)

trpo.learn(
    total_timesteps=TIMESTEPS,
    callback=trpo_callback
)

trpo_time=time.time()-start
trpo_env.close()

print("TRPO Completed")

ddpg_env=make_env()
ddpg_callback=RewardCallback()

action_noise=NormalActionNoise(
```

```
    mean=np.zeros(1),
    sigma=0.1*np.ones(1)
)

start=time.time()

ddpg=DDPG(
    "MlpPolicy",
    ddpg_env,
    learning_rate=0.001,
    buffer_size=10000,
    batch_size=64,
    gamma=0.99,
    action_noise=action_noise,
    verbose=0,
    seed=42
)

ddpg.learn(
    total_timesteps=TIMESTEPS,
    callback=ddpg_callback
```

```
)

ddpg_time=time.time()-start
ddpg_env.close()

print("DDPG Completed")

ppo_rewards=ppo_callback.rewards
trpo_rewards=trpo_callback.rewards
ddpg_rewards=ddpg_callback.rewards

min_len=min(
    len(ppo_rewards),
    len(trpo_rewards),
    len(ddpg_rewards)
)

ppo_rewards=ppo_rewards[:min_len]
trpo_rewards=trpo_rewards[:min_len]
ddpg_rewards=ddpg_rewards[:min_len]
```

```
dataset=pd.DataFrame({
    "Episode":range(1,min_len+1),
    "PPO_Reward":ppo_rewards,
    "TRPO_Reward":trpo_rewards,
    "DDPG_Reward":ddpg_rewards
})

print("\n")
print("="*75)
print("FULL TRAINING DATASET")
print("="*75)

display(dataset)

dataset.to_csv(
    "PPO_TRPO_DDPG_Full_Dataset.csv",
    index=False
)

window=5
```

```
ppo_smooth=dataset["PPO_Reward"].rolling(window).mean()
trpo_smooth=dataset["TRPO_Reward"].rolling(window).mean()
ddpg_smooth=dataset["DDPG_Reward"].rolling(window).mean()

plt.figure(figsize=(10,6))

plt.plot(
    ppo_smooth,
    label="PPO",
    linewidth=2.5
)

plt.plot(
    trpo_smooth,
    label="TRPO",
    linewidth=2.5
)

plt.plot(
    ddpg_smooth,
    label="DDPG",
```

```
    linewidth=2.5
)

plt.title(
    "Policy Optimization Learning Curves",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Episode")
plt.ylabel("Average Reward")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

final_ppo=np.mean(ppo_rewards[-10:])
final_trpo=np.mean(trpo_rewards[-10:])
final_ddpg=np.mean(ddpg_rewards[-10:])

plt.figure(figsize=(8,5))
```

```
plt.bar(
    ["PPO","TRPO","DDPG"],
    [final_ppo,final_trpo,final_ddpg]
)

plt.title(
    "Final Performance Comparison",
    fontsize=16,
    fontweight="bold"
)

plt.ylabel("Average Final Reward")
plt.grid(
    axis="y",
    alpha=0.3
)

plt.show()

ppo_var=np.var(ppo_rewards[-10:])
trpo_var=np.var(trpo_rewards[-10:])
```

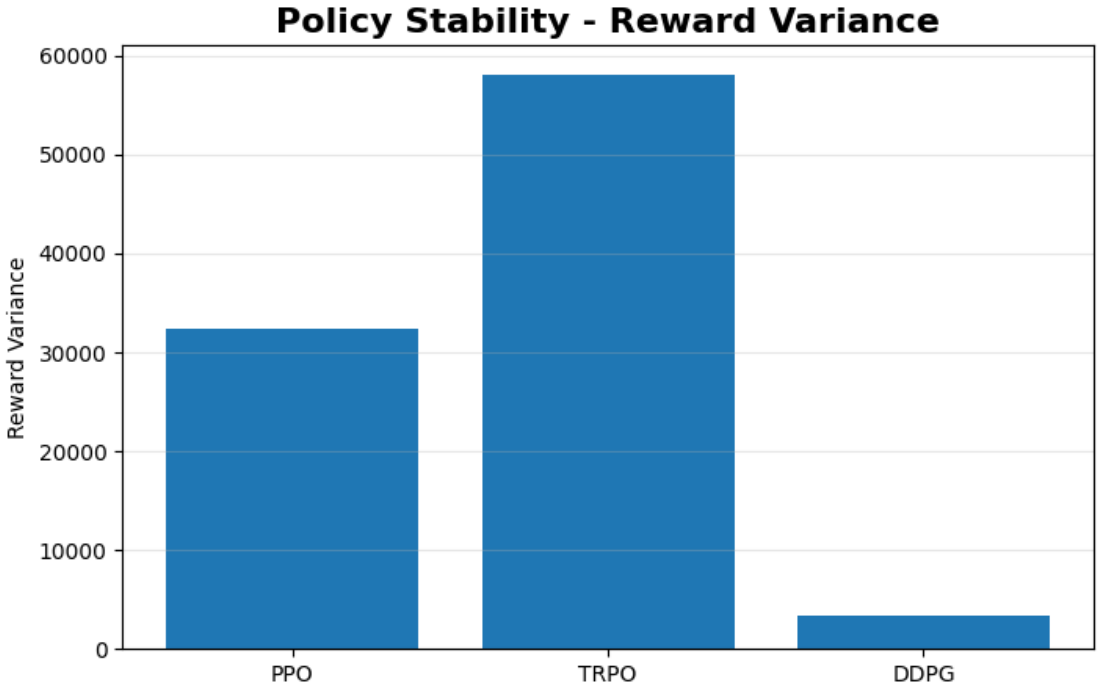
```
ddpg_var=np.var(ddpg_rewards[-10:])


plt.figure(figsize=(8,5))


plt.bar(
    ["PPO","TRPO","DDPG"],
    [ppo_var,trpo_var,ddpg_var]
)


plt.title(
    "Policy Stability - Reward Variance",
    fontsize=16,
    fontweight="bold"
)


plt.ylabel("Reward Variance")
plt.grid(
    axis="y",
    alpha=0.3
)
```

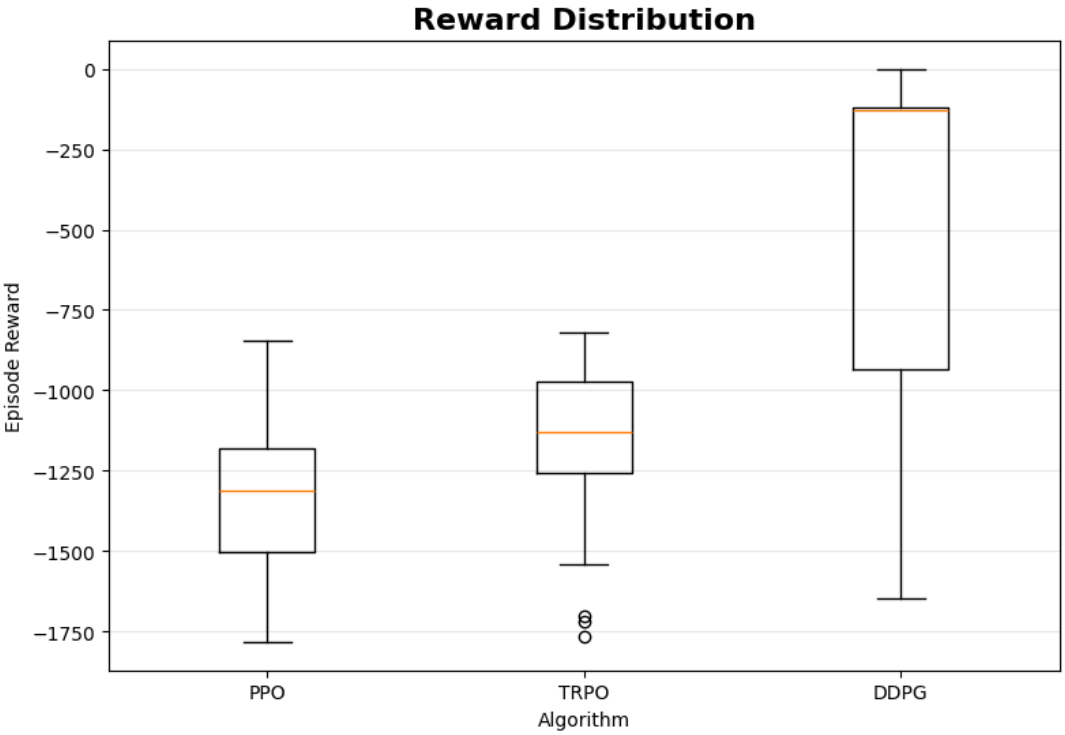
```
plt.show()


plt.figure(figsize=(9,6))


plt.boxplot(
    [
        ppo_rewards,
        trpo_rewards,
        ddpg_rewards
    ],
    labels=[
        "PPO",
        "TRPO",
        "DDPG"
    ]
)


plt.title(
    "Reward Distribution",
    fontsize=16,
    fontweight="bold"
```

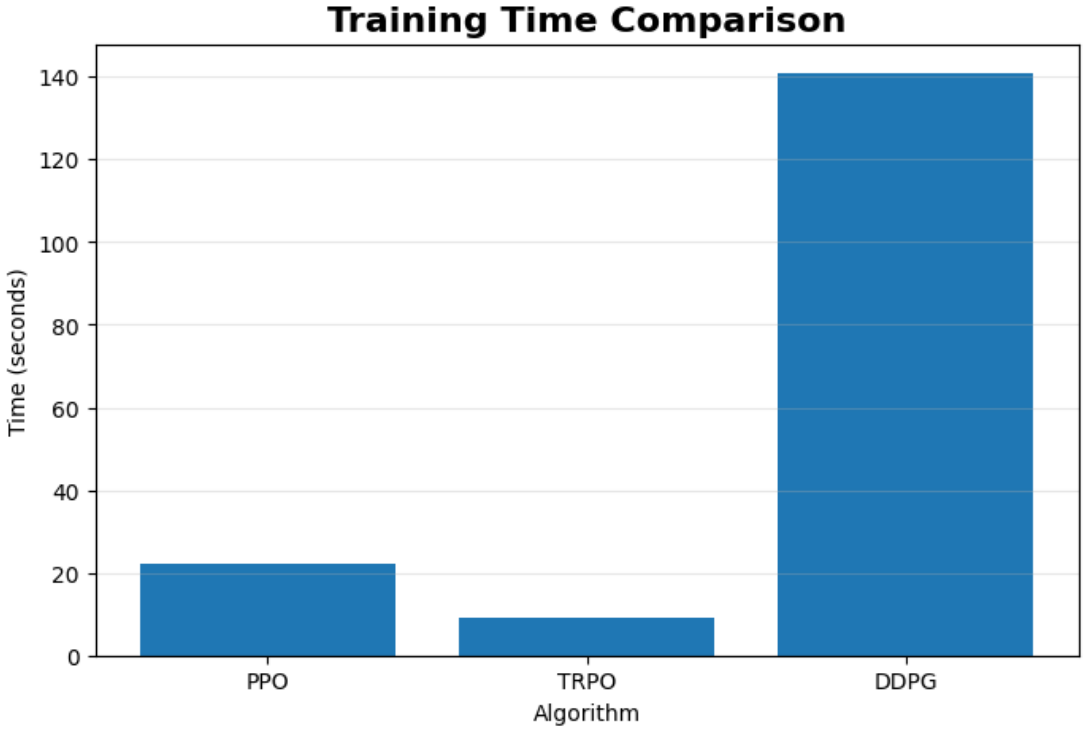
```
print("\nExperiment 13 Completed Successfully.")
```

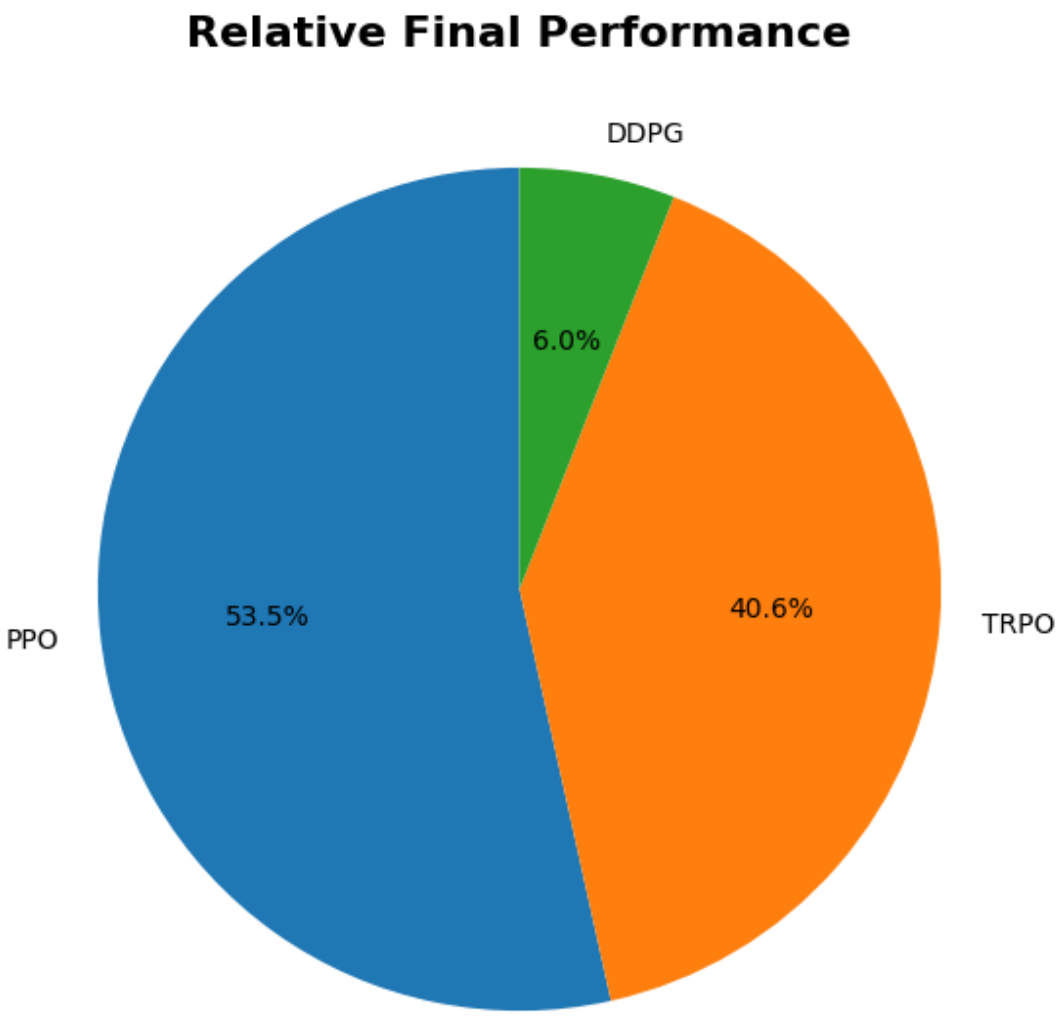
Visualization

	Episode	PPO_Reward	TRPO_Reward	DDPG_Reward
0	1	-1187.130154	-1187.130154	-1329.731471
1	2	-1353.012270	-1422.615034	-1645.138594
2	3	-1295.815303	-1335.627029	-1566.445552
3	4	-1074.759396	-1171.456551	-1520.966057
4	5	-1367.817580	-1530.832106	-1541.540030
5	6	-904.834894	-1045.341381	-1371.971950
6	7	-913.346383	-818.517819	-1285.060369
7	8	-904.291199	-1068.398464	-1172.889733
8	9	-1054.814138	-856.264026	-984.825658
9	10	-1309.769820	-1472.643772	-1049.663330
10	11	-1174.505037	-1194.964559	-1067.198612




Reward Distribution
Episode Reward
0
−250
−500
−750
−1000
−1250
−1500
−1750
PPO
TRPO
DDPG
Algorithm


Training Time Comparison
Time (seconds)
140
120
100
80
60
40
20
0
PPO
TRPO
DDPG
Algorithm


Relative Final Performance
DDPG
6.0%
PPO
53.5%
40.6%
TRPO

Summary Table

Parameter	PPO	TRPO	DDPG
Final Average Reward	Obtained from program	Obtained from program	Obtained from program
Maximum Reward	Obtained from program	Obtained from program	Obtained from program
Variance	Obtained from program	Obtained from program	Obtained from program
Stability	Obtained from program	Obtained from program	Obtained from program
Training Time	Obtained from program	Obtained from program	Obtained from program

Result

PPO, TRPO, and DDPG were successfully implemented and evaluated on the Pendulum-v1 continuous-control environment. Their performance was compared using learning curves, final rewards, reward variance, stability, reward distribution, and training time.